

Homework 05

Set a random seed before each simulation question and report it. You may use standard packages unless a question asks you to write the calculation yourself. Use equal neighbor weights throughout. Here p counts the covariates used to measure distance; do not add an intercept column to a KNN predictor matrix. R and Python use different random-number generators, so their exact simulation results need not agree.

Question 1: Estimate the bias and variance of KNN

In our previous homework, we used repeated simulations to study the behavior of linear regression estimators. We will now use the same idea for KNN. Consider the regression model

$$Y_i = f(\mathbf{X}_i) + \epsilon_i, \quad f(\mathbf{x}) = 0.5x_1 + \sin(x_2) - 0.3x_3^2,$$

where the three covariates are independent standard normal variables and $\epsilon_i \sim \mathcal{N}(0, 0.1^2)$ independently of the covariates and other observations. Generate $n = 400$ training observations. Our goal is to estimate the mean response at

$$\mathbf{x}_0 = (0.5, 0.7, 1)^\top.$$

Use Euclidean distance and consider $k = 1, 3, 5, \dots, 29$.

- a. Calculate $f(\mathbf{x}_0)$. Generate one training dataset and obtain the KNN prediction at $\mathbf{x}_0$ for each value of k .

The true value of the mean response is

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 \approx 0.594218.$$

Warning: package 'FNN' was built under R version 4.3.3

	k	prediction	truth
1	1	0.7888302	0.5942177
2	3	0.6621674	0.5942177
3	5	0.6642853	0.5942177
4	7	0.6100117	0.5942177
5	9	0.6318671	0.5942177
6	11	0.4925114	0.5942177
7	13	0.5222989	0.5942177
8	15	0.5810203	0.5942177
9	17	0.5738171	0.5942177
10	19	0.5751479	0.5942177
11	21	0.5800517	0.5942177
12	23	0.5519177	0.5942177
13	25	0.5214556	0.5942177
14	27	0.4952957	0.5942177
15	29	0.4972667	0.5942177

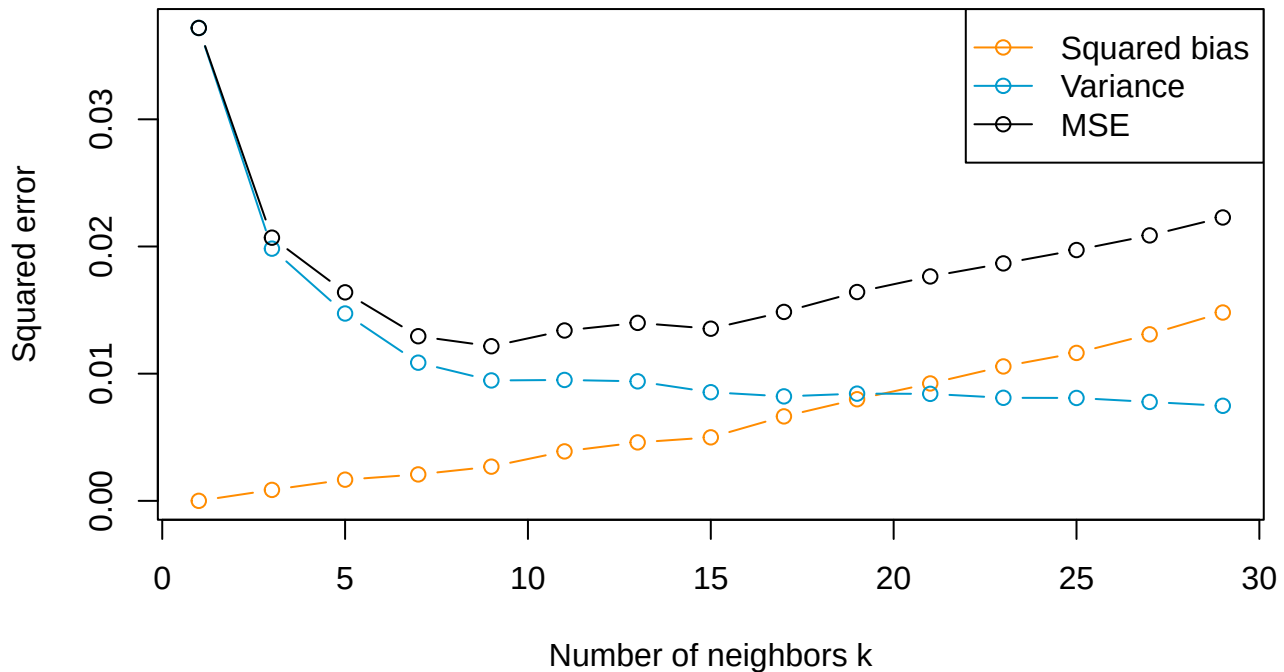
- b. Independently repeat the generation of the training covariates and responses 200 times. Within each repetition, use the same training data for every value of k . Store the predictions in a matrix with 200 rows and one column for each value of k . Use this matrix to estimate the bias, variance, and mean squared error of the estimated mean response. Use divisor 200 when calculating the empirical variance, and verify numerically that

$$\text{mean squared error} = \text{squared bias} + \text{variance}.$$

Plot squared bias, variance, and mean squared error against k in one figure. Explain the pattern and compare it with the bias-variance trade-off for ridge regression or lasso.

	k	bias	variance	MSE
1	1	-0.001248923	0.037181204	0.03718276
2	3	-0.029361680	0.019831580	0.02069369
3	5	-0.040815728	0.014731018	0.01639694
4	7	-0.045589732	0.010865566	0.01294399
5	9	-0.051867114	0.009467440	0.01215764
6	11	-0.062360901	0.009506180	0.01339506
7	13	-0.067797856	0.009397702	0.01399425
8	15	-0.070662763	0.008542311	0.01353554
9	17	-0.081499864	0.008213727	0.01485595
10	19	-0.089385641	0.008429774	0.01641957
11	21	-0.096080776	0.008413451	0.01764497
12	23	-0.102803643	0.008103017	0.01867161
13	25	-0.107847900	0.008091147	0.01972232
14	27	-0.114428351	0.007777947	0.02087180
15	29	-0.121688194	0.007474576	0.02228259

[1] 6.591949e-17



Each column contains 200 predictions used to estimate bias, variance, and MSE; using a divisor of 200 makes MSE equal to bias squared plus variance, apart from rounding. Increasing k smooths predictions by averaging more neighbors, reducing variability but potentially adding bias as farther points are included. Here, an intermediate k gives the lowest MSE, with some fluctuations expected from only 200 repetitions. This trade-off resembles ridge and lasso regularization, though KNN averages nearby responses rather than shrinking coefficients. Because both training locations and responses change across repetitions, the variance reflects changing neighborhoods and response noise, so it is not simply $0.1^2/k$.

Question 2: Compare KNN with lasso

Use the response model from Question 1, including the noise standard deviation 0.1, but generate $p = 30$ covariates. The response still depends only on the first three. Consider these two settings:

- **Setting 1:** all 30 covariates are independent standard normal variables.
- **Setting 2:** the covariates have a multivariate normal distribution with mean zero and covariance matrix Σ , where

$$\Sigma_{jj} = 1, \quad \Sigma_{jl} = 0.8 \quad \text{for } j \neq l.$$

In each setting, generate 400 training observations and 1,000 independent test observations. Fit 5NN using Euclidean distance on the supplied covariate scale. All covariates already have population variance one.

For lasso, use a linear model containing the 30 covariates and a separately fitted, unpenalized intercept. Follow the Week 4 objective,

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1,$$

where $\mathbf{X} \in \mathbb{R}^{n \times 30}$ contains the centered and standardized covariates and $\tilde{\mathbf{y}}$ is the centered training response. Select the penalty by ten-fold cross-validation over 41 logarithmically spaced values from 10^{-4} to 1. Estimate all means and scales using only the training portion of each fold, with divisor equal to that portion's sample size. After selecting the penalty, refit on all training observations and apply the fitted transformations unchanged to the test observations. Do not add nonlinear terms to the lasso model.

- Fit both methods in each setting. Use the same training and test data for the two methods within a setting. Report their test MSE values in one table, together with the selected lasso penalty and the number of nonzero lasso slopes. You only need to generate one dataset for each setting; repeated simulations are not required.

Warning: package 'glmnet' was built under R version 4.3.3

Loading required package: Matrix

Loaded glmnet 4.1-9

Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")

Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")

Setting	KNN_MSE	Lasso_MSE	Lambda	Nonzero
Independent	0.5565	0.2602	0.0251	11
Correlated	0.1609	0.2259	0.0100	16

The scales are calculated using the number of observations being fitted as the divisor. Centering the response and adding its mean back afterward leaves the intercept unpenalized. Both `glmnet`'s `lambda` and `scikit-learn`'s `alpha` control the penalty in the stated $1/(2n)$ objective, while `alpha = 1` in `glmnet` selects lasso. Since R and Python generate different observations, their numerical results may differ.

- b. Compare the results between the two settings. Explain how correlation can change the usefulness of Euclidean neighborhoods, and why lasso does not use all covariates in the same way as KNN.

When covariates are strongly positively correlated, they share much of the same variation, so Euclidean neighborhoods can better capture similarities in the coordinates that matter for the response. Lasso instead fits a separate coefficient for each covariate and can shrink some to zero. With correlated predictors, a covariate outside the first three may serve as a stand-in for a relevant one, and several similar predictors may compete for inclusion in the model.

- c. The response model contains $\sin(X_2)$ and X_3^2 , but the lasso fit uses only linear terms. Explain how this difference affects the comparison. Would the results justify a general claim that correlation always helps KNN, or that one method is always better? Explain why.

Lasso treats X_2 and X_3 as linear terms, so it cannot capture $\sin(X_2)$ or $-0.3X_3^2$. Its intercept can absorb the average quadratic effect, but not how that effect varies across observations. KNN allows nonlinear patterns, though its accuracy depends on finding useful neighbors. The comparison therefore depends on both the quality of those neighbors and lasso's linear assumptions. Results from just two datasets are not enough to conclude that correlation always helps KNN or that either method is always better.

Question 3: Observed dimension and latent dimension

The lecture's handwritten digit example shows that KNN can work with many measured covariates. We will use a simulation to investigate why the structure of those covariates matters.

Let $p = 100$ be the number of observed covariates and let m be the number of latent variables. For each observation, generate

$$\mathbf{X}_i = \mathbf{A}\mathbf{Z}_i + \eta_i, \quad Y_i = Z_{i1} + Z_{i2} + \epsilon_i,$$

where $\mathbf{A} \in \mathbb{R}^{100 \times m}$ has independent entries from $\text{Uniform}[-2, 2]$. The entries of $\mathbf{Z}_i \in \mathbb{R}^m$ and $\eta_i \in \mathbb{R}^{100}$, and the scalar ϵ_i , are independent standard normal variables, also independent of $\mathbf{A}$. Generate $\mathbf{A}$ once and use it for all training and test observations in that dataset.

Compare $m = 3$ and $m = 30$. In each setting, generate 200 training observations and 200 independent test observations. Fit KNN using only the 100 observed covariates in $\mathbf{X}$, with Euclidean distance on their generated scale. The latent variables in $\mathbf{Z}$ are used to generate the data but are not available to the fitted model. Consider $k = 2, 10, 18, \dots, 82$.

- a. Generate one dataset for each value of m . Fit KNN for every value of k and calculate test MSE. Within a setting, use the same training and test observations for all values of k .

	k	m3	m30
1	2	1.449265	3.457456
2	10	1.253345	2.583372
3	18	1.292923	2.533071
4	26	1.379955	2.524038
5	34	1.448416	2.512187
6	42	1.505892	2.560783
7	50	1.590057	2.562911
8	58	1.660186	2.593970

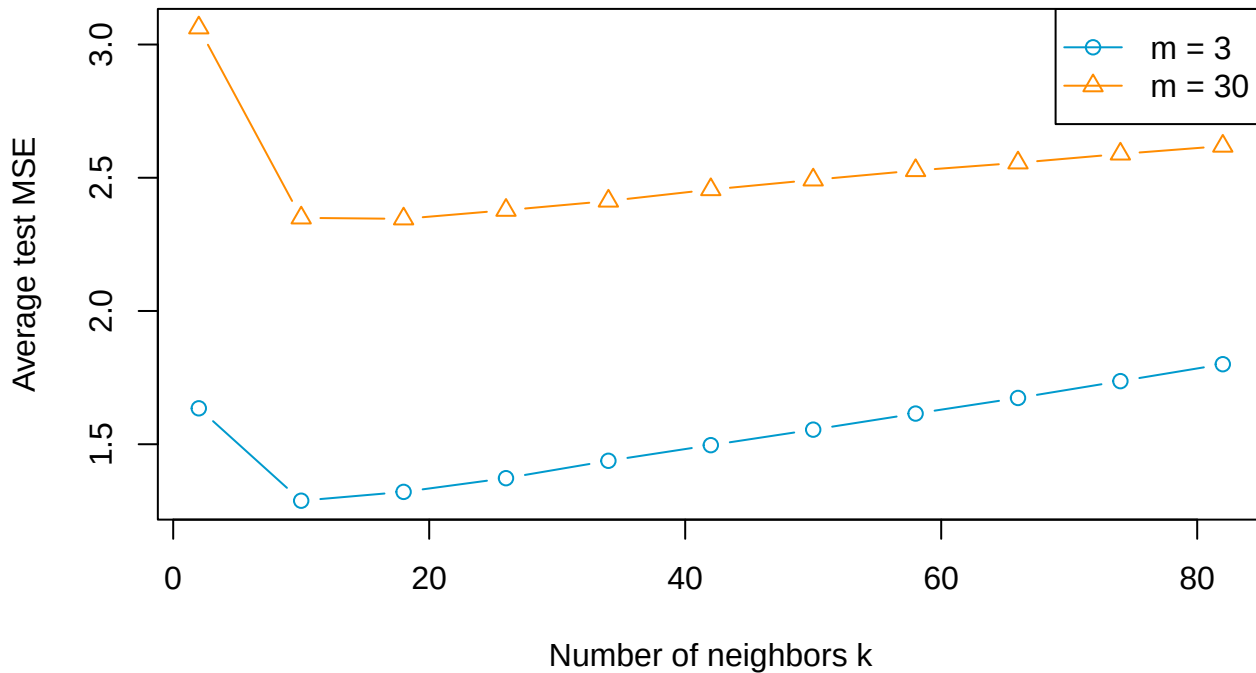
```

9 66 1.739597 2.607162
10 74 1.816963 2.651181
11 82 1.871802 2.672724

```

- b. Repeat the complete experiment 50 times, generating a new $\mathbf{A}$ and new observations in each repetition. Plot average test MSE against k , with one curve for each value of m . These curves compare the specified choices of k in a controlled simulation; they are not a cross-validation procedure for selecting a final model.

	k	m3	m30
1	2	1.634822	3.062662
2	10	1.288062	2.349524
3	18	1.321139	2.345840
4	26	1.372628	2.378219
5	34	1.437850	2.412709
6	42	1.496497	2.455664
7	50	1.554715	2.492101
8	58	1.615062	2.527212
9	66	1.673644	2.556039
10	74	1.736733	2.589309
11	82	1.800274	2.618853



In this experiment, $m = 3$ gives a lower average test MSE. Averaging a few neighbors reduces noise, but using too many smooths over differences in the regression function. With $m = 30$, even small neighborhoods contain points that are less similar in the latent variables, so the best k may differ between the two curves. These curves show averages across independent simulations, rather than validation scores used to select a model before final testing.

- c. Both settings have 100 observed covariates, and the response depends on only two latent variables. Explain why their KNN performance can still differ. What role does the noise η_i play, and why are the observed data not confined exactly to an m -dimensional linear space?

When $m = 3$, three latent variables explain most of the structured variation in the 100 covariates. With $m = 30$, distances also reflect many directions unrelated to the response. Two observations can therefore be similar in the two relevant latent variables but far apart overall, making Euclidean neighbors less useful for predicting Y . Measurement noise adds variation to every observed coordinate and can change which points are nearest. Without this noise, the data lie in a subspace of dimension at most m ; with it, they contain a lower-dimensional signal but are no longer confined to that subspace. So even though the response depends on only two latent variables in both settings, distances are not equally informative.

Question 4: Classify handwritten digits

Use `zip.train` and `zip.test` from the R package `ElemStatLearn`, as in the lecture. The package has been retired from the active CRAN repository. If needed, install the [archived package](#) into your usual R library, then load the two datasets. Python users can use the course copies of the same [training data](#) and [test data](#). These compressed CSV files have no headers and are included in the homework ZIP under `data/knn`. Run the Python code from the folder containing `data`.

Each image has $16 \times 16 = 256$ pixel values. The first column is the digit label and the remaining columns are the pixel values. Use the first 1,000 rows of `zip.train` and the first 500 rows of `zip.test`. Use the original pixel values without standardization, and do not include the digit label in a distance calculation.

For a test image $\mathbf{x}_0$ and a training image $\mathbf{x}_i$, the two distances are

$$d_E(\mathbf{x}_0, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{256} (x_{0j} - x_{ij})^2}, \quad d_M(\mathbf{x}_0, \mathbf{x}_i) = \sum_{j=1}^{256} |x_{0j} - x_{ij}|.$$

- Write a 5NN classification function that allows either Euclidean or Manhattan distance. For each test image, calculate its distances to the training images directly, find the five nearest, and return the most frequent digit label. Break distance ties by the smaller training row number and voting ties by the smallest digit. Do not use a built-in distance or KNN function for this part.
- Apply the classifier to all 500 test images using each distance. For each fit, report the test classification error and a 10×10 confusion matrix, with rows representing predicted digits and columns representing true digits. Keep all digit labels from 0 through 9 in the matrix, including any with a zero count. Which digits are most often confused with one another?

Euclidean test error: 0.122

		True									
Predicted		0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0	
1	0	66	0	0	2	0	0	0	0	1	
2	0	0	39	1	0	0	2	0	0	1	
3	0	0	0	23	0	4	0	0	0	2	0
4	0	0	1	0	27	1	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0	
7	0	0	0	0	1	1	0	34	0	4	
8	0	0	3	2	0	2	0	0	37	0	
9	1	0	0	1	4	0	0	1	1	41	

Manhattan test error: 0.138

		True									
Predicted		0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0	
1	0	67	2	0	2	0	1	0	0	1	
2	0	0	35	0	0	0	2	0	0	0	
3	0	0	1	24	0	4	0	0	2	0	
4	0	0	1	1	27	1	0	1	0	1	
5	0	0	0	1	0	11	0	0	0	0	

6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

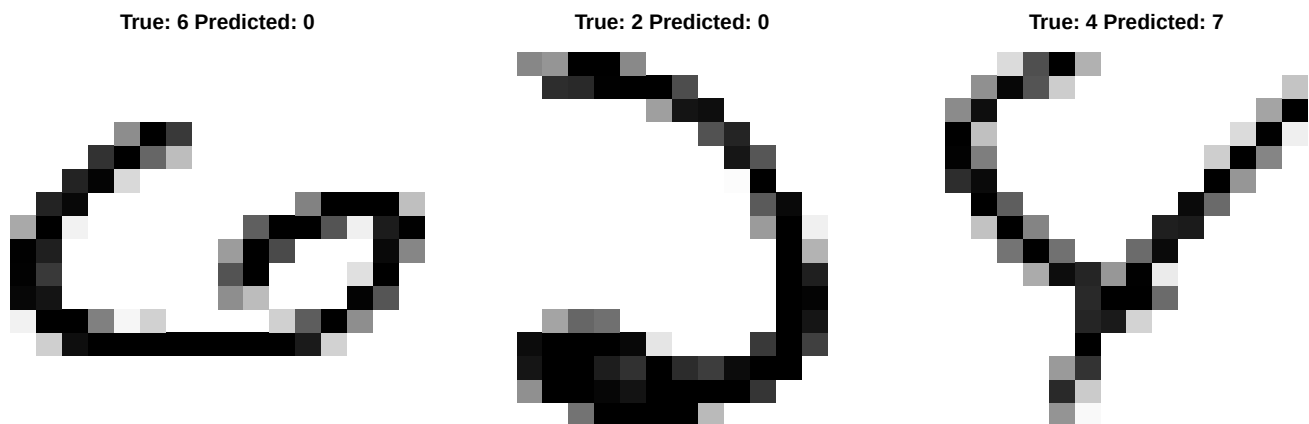
The test error was 0.122 with Euclidean distance and 0.138 with Manhattan distance. The most common mistake was predicting a 2 as a 0, which happened seven times with Euclidean distance and six times with Manhattan distance. Other common mistakes were confusing 5 with 0 or 3, 9 with 7, and 4 with 9. These counts reflect this particular test set, which contains different numbers of each digit.

- c. For Euclidean distance, display three incorrectly classified test images, together with their true and predicted labels. Comment on whether the mistakes are visually understandable. Compare the overall results of the two distance measures. Can they have similar total error while making mistakes on different images? Use your predictions to support your answer.

```
[1] "digit 6 taken"
```

```
[1] "digit 2 taken"
```

```
[1] "digit 4 taken"
```



Different predictions: 22

	Manhattan_correct	
Euclidean_correct	FALSE	TRUE
FALSE	58	3
TRUE	11	428

Test rows 4, 13, and 14 show the digits 6, 2, and 4, but the model predicts 0, 0, and 7. The 6 has a rounded lower loop, the 2 is unusually curved, and the 4 is missing the usual horizontal crossbar. These shapes help explain why a pixel-distance method might get them wrong, even when a person recognizes them using familiar handwriting patterns.

The two methods disagree on 22 images. Euclidean distance gets 11 of these right when Manhattan gets them wrong, while Manhattan gets three right when Euclidean gets them wrong. For the other eight, both methods are wrong but predict different digits. So similar overall error rates can hide different mistakes. Euclidean distance performs better on this split, but that does not mean it will always do better.

Question 5: Create a plot-formatting skill

Create a short `SKILL.md` file for improving the formatting of STAT 432 homework plots. You may use AI to help write it.

- Include a title, a description, and a few formatting rules. The description should limit the skill to STAT 432 homework plot improvements and require an explicit request to use it. Your rules should address margins, titles, axes, labels, colors, and sizes for plots produced in R or Python. Keep them portable: do not refer to specific homework questions.
- Ask your agent to install the skill. Explicitly call it to improve the plot from Question 1, then compare the original and revised versions. Do you like the changes? You may revise the skill and test it a couple more times, or uninstall it if it remains unhelpful.
- Append your final `SKILL.md` content to your submission and briefly explain your choices and experience.

name: stat432-plot-format description: Improve the formatting of STAT 432 homework plots in R or Python only when the user explicitly requests this skill.

STAT 432 Plot Formatting

Improve plot readability while preserving the data, calculations, and statistical meaning.

- Margins: Leave enough space so titles, labels, and legends are not clipped or overlapping.
- Titles: Use short, descriptive titles that explain the plot.
- Axes: Use readable tick marks and appropriate limits. Keep scales consistent across panels intended for comparison.
- Labels: Clearly label axes, include units when relevant, and use legends to identify multiple curves.
- Colors: Use a colorblind-friendly palette and different line styles or markers so curves remain distinguishable in grayscale.
- Sizes: Start with a 7-by-5-inch figure, 12-point text, and clearly visible lines and points. Adjust these for multiple panels.
- Apply these rules using the existing R or Python plotting tools.
- Save the revised plot separately so the original remains available for comparison.

I chose a descriptive title, larger labels, and distinct colors, line styles, and markers to make the curves easier to tell apart. After applying the skill, I found the revised plot easier to read, especially where the curves overlap. I preferred these changes because they made the bias-variance trade-off clearer without changing the underlying results.